

AI / LLM / Agentic Threat Modelling Runbook

A facilitator's checklist for running design-time threat modelling sessions on AI-powered systems

Copyright 2026 Brett Crawley. This runbook was created by Brett Crawley **Version 0.1 Creative Commons Attribution-ShareAlike 4.0**

How to use this runbook

This is a working document for the person facilitating a threat modelling session, not a reference for the team attending it. It assumes you have read the companion document (*Threat Modelling AI, LLM and Agentic Systems*) and are comfortable with the underlying frameworks: OWASP LLM Top 10 (2025), OWASP Agentic Top 10 (ASI01 to ASI10), LINDDUN, T.R.I.M., and the Threat Modeling Manifesto's four questions.

The runbook is organised into five phases:

1. **Pre-session:** what you do before anyone walks in the room.
2. **Phase 1: Frame the system:** "What are we working on?"
3. **Phase 2: Walk-through the threats:** "What can go wrong?"
4. **Phase 3: Agree the mitigations:** "What are we going to do about it?"
5. **Post-session:** "Did we do a good enough job?" and keeping the threat model alive.

Each phase has a checklist, a set of facilitator prompts, and a time box. Total session time is between 90 minutes (small feature) and a full day (a net-new agentic system like DevAssist). Do not try to do it all in one meeting for anything complex. Better to run three focused two-hour sessions than one death march.

Note to facilitators: your job is not to be the expert on every threat. Your job is to make sure the team covers the entire attack surface, argues honestly, and leaves with design changes. If you find yourself doing all the talking, you are in the wrong role.

Pre-session

Facilitator checklist

- ☐ You know which feature or system is being modelled and at what lifecycle stage (greenfield design, pre-launch review, incident response, quarterly refresh).
- ☐ You have a one-paragraph description of what the system does, written by someone on the team, not by you.
- ☐ You know who owns the system and who will own the mitigations. If you cannot name the owner, the session will produce an orphan document.
- ☐ The right people are invited: at minimum, the architect, a senior engineer who will actually build it, a product owner, and whoever handles privacy. For agentic systems, add whoever manages identity and credentials. For

anything customer-facing, add someone from the incident response side.

- ☐ You have blocked enough time. As a rule of thumb: 90 minutes minimum, four hours for a system with any agentic or MCP components, full day for a multi-agent system with shared memory.
- ☐ You have whiteboarding tools (physical or digital). Drawing the DFD is non-negotiable.
- ☐ You have a lightweight way to capture threats, owners, and deadlines. A spreadsheet is fine. A backlog is better. A Word document is marginally better than nothing at all.
- ☐ If the team has not threat modelled before, you have set expectations in writing beforehand: this is not a security audit, nobody is being graded, the goal is better design.
- ☐ For AI systems specifically, the team has read at least a short primer on the OWASP LLM Top 10 and the Agentic Top 10. Do not run the session cold on AI-literate frameworks; pre-read is mandatory or you will spend the first hour explaining prompt injection.

Materials to bring

- A one-page explainer on the adversarial subspace problem and decision-boundary transferability, to circulate to attendees beforehand. These two concepts are the single biggest predictor of whether a session produces structural mitigations or theatre. If attendees arrive believing model guardrails are the defence, the session will produce the wrong answers. Pre-reading is not optional.
- A printed or digital copy of the OWASP LLM Top 10 and Agentic Top 10 entries, one line each.
- The seven-question MCP review template (see Phase 2).
- The T.R.I.M. four-card prompts (Transfer, Retention/Removal, Inference, Minimisation).
- A copy of the LINDDUN categories if privacy is in scope.
- A blank DFD canvas.
- The mitigation-to-threat traceability matrix template.
- Optional: the F-Secure Elevation of Privacy deck and the Shostack Elevation of Privilege deck if you are running the session with a team new to threat modelling. The cards accelerate participation; they are not mandatory.

Questions to answer before the session starts

You should be able to answer these yourself, or you are not ready to facilitate:

- What is the sensitivity of the data the system touches? Public, internal, personal, special-category?
- Is the system customer-facing or internal-only?
- What is the blast radius if the system does the worst thing it could do?
- What regulatory regimes apply? GDPR, EU AI Act, sector-specific (HIPAA, PCI DSS, MiFID II, NIS2)?
- Is this greenfield or are you threat modelling something already in production? Late-stage threat models are harder and need explicit framing as "what do we fix before we make it worse".

Phase 1: Frame the system (What are we working on?)

Time box: 30–45 minutes.

The goal of this phase is to produce a DFD and a shared understanding of what is actually being built. Most sessions fail here because the team jumps to threats before agreeing on the system. Resist this.

Checklist

- ☐ A DFD exists on the whiteboard.
- ☐ Every actor is on the diagram (users, agents, external services, attackers).
- ☐ Every data store is on the diagram (databases, vector stores, memory, caches, logs).
- ☐ Every external service is on the diagram (hosted LLM vendor, MCP servers, third-party APIs).
- ☐ Trust boundaries are drawn as dashed lines, not implied.
- ☐ Data classifications are marked on each flow: public, internal, personal, special-category, secrets.
- ☐ The team agrees the diagram reflects reality, not the ideal version that will exist in three months.

Facilitator prompts

Use these verbatim. They are deliberately basic; the magic is in getting the team to answer them honestly.

- "Who uses this system? Walk me through a real user journey, start to finish."
- "Where does the data come from? Name every source."
- "Where does the data go? Name every destination."
- "Which of these boxes are under our control, and which are not?"
- "For each arrow crossing into something we do not control, what stops bad things from flowing back?"
- "If an attacker sat at point X, what could they see or do?"

AI-specific framing questions

These are the questions a generic threat modelling session would miss.

- "What is the foundation model, and who is it hosted by? Every prompt we send goes to them."
- "Is there a RAG pipeline? What does it index, how often, and who controls the source documents?"
- "Is there long-term memory? Who can write to it, who can read it, and is it partitioned per user?"
- "Which MCP servers or plugins are installed? Who wrote them, how are they updated, what credentials do they inherit?"
- "Are there multiple agents? Who orchestrates them, and do they trust each other by default?"
- "What can the agent actually do? List every tool and every action with a side effect."
- "What does a 'long session' look like for this system? Forty turns? Four hundred? A full working day?"

Red flags at this phase

If any of these come up, stop and fix them before moving on.

- The team cannot agree on the diagram. This means the system is not designed yet, and you are threat modelling something that does not exist.
- The diagram has no trust boundaries. Either you are missing them or the system has no external interfaces, and the latter is almost never true.
- The diagram has an MCP client but no list of installed servers. Get the list before proceeding.
- Someone says "the model takes care of that". The model does not take care of anything security-relevant. Write that threat down now.
- Someone says "we just use the vendor's safety features". The vendor's safety features are the first thing attackers target. Also write this down.

Deliverable

A DFD, a list of assets, a list of actors, a list of trust boundaries, and a list of data classifications. No threats yet. Resist the urge to jump ahead; it will make Phase 2 faster.

Phase 2: Walk-through the threats (What can go wrong?)

Time box: 60–180 minutes, depending on complexity.

This is the longest phase. The structure here is four passes, each using a different lens, walking through the DFD each time. Do not try to brainstorm threats from scratch. The lenses exist so you do not miss things.

For each threat identified, capture:

1. Threat name and ID (map to OWASP LLM Top 10 or Agentic Top 10 where possible).
2. Plain-English example specific to this system, not a generic one.
3. Which component(s) on the DFD it affects.
4. Likelihood and impact, rough scale (low/medium/high is fine; resist the urge to calculate a CVSS score).
5. Owner for the mitigation decision.

Pass 1: Adversarial threats (LLM Top 10 and Agentic Top 10)

Time box: 30–60 minutes.

Walk-through each OWASP entry against the DFD. For each one, ask the team: "Does this apply to our system? If yes, give me an example. If no, tell me why not."

The "why not" is as important as the "yes". It forces the team to justify exclusions and catches assumptions.

LLM Top 10 checklist

- ☐ **LLM01 Prompt Injection.** Walk-through every input channel: user prompt, RAG, tool outputs, memory, agent-to-agent messages. For each, ask "what stops a hostile instruction reaching the model through this path?"
- ☐ **LLM02 Sensitive Information Disclosure.** Ask: "What personal data or secrets could the model emit? What stops it?" Walk-through both training-time leakage and retrieval-time leakage.
- ☐ **LLM03 Supply Chain.** List every external dependency: base model, fine-tunes, adapters, libraries, MCP servers, prompt templates. For each, ask "what happens if this gets compromised tomorrow?"
- ☐ **LLM04 Data and Model Poisoning.** Ask: "Who can write to the training data, the fine-tuning data, or the RAG corpus? What review happens?"
- ☐ **LLM05 Improper Output Handling.** Walk-through every downstream consumer of model output: the renderer, the tool caller, the logger, the next agent. Ask: "What does this consumer assume about the trustworthiness of the model's output?"
- ☐ **LLM06 Excessive Agency.** For each tool: "Does the agent need this tool for the stated task? Does it need the permissions it has? Does it need autonomy, or should a human confirm?"
- ☐ **LLM07 System Prompt Leakage.** Ask: "What is in the system prompt that we would not want a user to see? If anything, move it out."

- ☐ **LLM08 Vector and Embedding Weaknesses.** For any RAG system: "Is access control enforced at retrieval or just at the UI? Is the store tenant-partitioned? Who can add content?"
- ☐ **LLM09 Misinformation.** Ask: "Where in the workflow does the model output become the basis for a decision or an action? What happens if it is confidently wrong?"
- ☐ **LLM10 Unbounded Consumption.** Ask: "What is the maximum cost a single malicious user (or a single bug) could run up in 24 hours? In a weekend?"

Agentic Top 10 checklist

Skip if the system has no agentic components. For anything with tool use, long-term memory, or multi-agent coordination, all ten apply.

- ☐ **ASI01 Agent Goal Hijack.** "What is the agent trying to do, and what could redirect it?"
- ☐ **ASI02 Tool Misuse and Exploitation.** "For each legitimate tool, what is the worst thing it could do if invoked with attacker-controlled arguments?"
- ☐ **ASI03 Identity and Privilege Abuse.** "What credentials does each agent hold? Are they scoped per task or shared? Who is accountable when they are misused?"
- ☐ **ASI04 Agentic Supply Chain.** "Who wrote the orchestration framework, the agent personas, the prompt templates, the MCP servers?"
- ☐ **ASI05 Unexpected Code Execution.** "Is there any path by which the agent can execute arbitrary code? Where is it sandboxed? What is the egress policy?"
- ☐ **ASI06 Memory and Context Poisoning.** "Who can write to the memory store? Is provenance attached? Can one user's session affect another's?"
- ☐ **ASI07 Inter-Agent Communication Exploitation.** "Do the agents authenticate to each other? Do they validate structured messages? Do they trust return values?"
- ☐ **ASI08 Cascading Failures.** "If agent A returns a wrong answer, what happens in agents B, C, and D? Where are the circuit breakers?"
- ☐ **ASI09 Human-Agent Trust Exploitation.** "What does the agent produce that a human will act on without checking? What provenance does the UI show?"
- ☐ **ASI10 Rogue Agents.** "How would you know if an unauthorised agent was running in this environment? How would you kill it?"

Pass 2: Structural hazards

Time box: 15 minutes.

These are the failures that happen without an attacker and that the OWASP lists miss. Short pass, but do not skip it.

- ☐ **Context rot.** "What is the longest realistic session for this system? At that length, which security-critical instructions are at risk of being attentionally lost? Which tool constraints, which authorisation scopes, which content rules?"
- ☐ **Hallucination as a structural failure.** "Where in the workflow does the model generate content that will be read, indexed, persisted, or acted upon without verification?"
- ☐ **Transferable decision boundaries.** "What security guarantees are we relying on the model itself to provide? If the answer is 'robustness against adversarial inputs', write that down as an assumed-broken control and move the guarantee to a deterministic layer."

- ☐ **Geometric adversarial attacks.** "Does this system expose a classifier or matcher whose output gates a decision? If yes, can an attacker probe it with queries? Is the hard label the only signal returned? Does the model use non-Euclidean geometry (hyperbolic, spherical, angular-margin)? If yes, is our adversarial testing matched to that geometry?"
- ☐ **Adversarial subspace problem.** "Are we using a prompt library, blocklist, or pattern-match as a primary defence against prompt injection or jailbreak? If yes, flag as a subspace-problem failure and escalate the mitigation to structural. Is our security guarantee architectural (downstream deterministic enforcement) or statistical (guardrails that pattern-match)?"

Pass 3: Privacy threats (LINDDUN + T.R.I.M. + GDPR)

Time box: 30–45 minutes if personal data is in scope. Skip if there is genuinely no personal data, but be sceptical; "no personal data" is rarely true.

Walk-through the DFD again, this time asking privacy questions. I find it works best to do T.R.I.M. first as an accessible warm-up, then LINDDUN for depth, then map to GDPR Article 5 at the end.

T.R.I.M. four-card walk-through

For each data flow carrying personal data on the DFD:

- ☐ **Transfer.** "Where is this personal data going next? Who is the recipient? Do they have a lawful basis? Is the data provenance-tagged so they can tell collected data from model-generated data?"
- ☐ **Retention/Removal.** "Where does this data end up persisted? For each store: can we locate and delete a specific individual's data, including model-generated statements about them, within 30 days of a request?"
- ☐ **Inference.** "What new personal data does the model create on this flow that was not in the input? Is any of it special-category under GDPR Article 9? What is our basis for creating it?"
- ☐ **Minimisation.** Two sub-questions: "Is the input minimal for the task?" and "Is the output minimal for the task? Is the model generating more personal data than the user asked for?"

LINDDUN seven-category walk-through

- ☐ **Linking.** "What links between data points does this system create that did not exist before?"
- ☐ **Identifying.** "Where does the system turn partial or pseudonymous data into identification?"
- ☐ **Non-repudiation.** "Does the system create utterances attributable to a real person that they could not credibly deny?"
- ☐ **Detecting.** "Does the system reveal membership in a dataset through the presence or absence of a response?"
- ☐ **Data Disclosure.** "Both flavours: real personal data leaking, and fabricated personal data being treated as real."
- ☐ **Unawareness / Unintervenability.** "Can the data subject find out what the system knows or claims about them? Can they correct it?"
- ☐ **Non-compliance.** "What GDPR articles does everything above engage, and can we satisfy them?"

GDPR Article 5 backstop

- ☐ **5(1)(a) lawfulness, fairness, transparency.** "Is every processing operation here lawful, fair, and disclosed in the privacy notice?"
- ☐ **5(1)(b) purpose limitation.** "Are we processing personal data only for the purposes we declared?"

- ☐ **5(1)(c) minimisation.** "Applied to both inputs and outputs?"
- ☐ **5(1)(d) accuracy.** "If the model generates inaccurate personal data, can we rectify it under Article 16?"
- ☐ **5(1)(e) storage limitation.** "Do we have defensible retention periods for every store containing personal data, including model outputs?"
- ☐ **5(1)(f) integrity and confidentiality.** "Mostly covered by Pass 1, but double-check."
- ☐ **Article 22.** "Does any model output feed an automated decision with legal or similarly significant effect? If yes, there are separate obligations."
- ☐ **EU AI Act overlay.** "What risk tier is this system under the AI Act, and what does that trigger?"

Pass 4: MCP-specific walk-through

Time box: 15 minutes per MCP server. Skip if there are no MCP or plugin components.

For each installed MCP server or equivalent plugin, walk-through the seven-question template:

- ☐ **Provenance.** Who wrote it? How was it installed? Is the version pinned? Is there an SBOM? Has anyone from our team read the source?
- ☐ **Process isolation.** Where does it run? As which user? What can it read on the host? What network egress does it have by default?
- ☐ **Credential scope.** What identities and tokens does it have access to? Are they scoped to its actual function, or does it inherit everything?
- ☐ **Tool surface.** What tools does it expose? Read the descriptions adversarially. Would you ship that text as a system prompt? You are.
- ☐ **Cross-server exposure.** What other servers share its client? What data could flow from high-trust to low-trust servers via the model?
- ☐ **Update model.** How is it updated? Will tomorrow's version run with the same trust as today's? Who re-reviews tool descriptions on update?
- ☐ **Observability and kill switch.** Are tool calls logged with their arguments and the tool description version? Can this server be disabled from one place in under five minutes?

If any server fails three or more of these questions, it should not be installed in any environment that matters. Flag it as a blocker, not a finding.

Deliverable at end of Phase 2

A list of threats, each with:

- An ID mapped to an OWASP entry, a LINDDUN category, a T.R.I.M. card, or a structural-hazard label.
- A system-specific example.
- The DFD component(s) affected.
- A likelihood/impact rating.
- A proposed owner.

No mitigations yet. Again, resist the urge to jump ahead.

Phase 3: Agree the mitigations (What are we going to do about it?)

Time box: 30–60 minutes.

The goal of this phase is to turn the threat list into a list of decisions. A threat without a decision is an unresolved item, not a finding.

Checklist

- ☐ Every threat has one of four decisions: **mitigate**, **accept**, **transfer**, or **avoid** (remove the feature).
- ☐ Every "mitigate" has a named control, a named owner, and a deadline.
- ☐ Every "accept" has explicit sign-off recorded at the appropriate level. Accepted risk without a named person accepting it is a dodge.
- ☐ Every "transfer" names the counterparty (vendor, insurer, downstream team) and verifies they actually accept the transfer. Do not assume.
- ☐ Every "avoid" is recorded as a design change so it does not silently return in the next iteration.

The "structural first" test

For each "mitigate" decision, ask: is this control as far left in the architecture as possible, or are we patching at the output?

If three out of four mitigations are runtime filters and guardrails, you are building defence in depth on a broken foundation. Push the decisions back to the design.

Facilitator prompts

- "What is the simplest change to the design that removes this threat entirely?"
- "If we could only implement one control for this, which would it be?"
- "Who specifically owns making this happen? Not the team, the person."
- "When will we know this is done? What does done look like?"
- "If we are accepting this risk, who signs off and what do they need to see?"

Categories of mitigation worth checking

Walk-through these quickly as a backstop. For each, ask "do we need one of these, and if so, where?"

- ☐ **Design-time structural controls:** least agency, scoped credentials, authorisation at the data layer, sandboxed code execution, MCP isolation, treat retrieved content as untrusted.
- ☐ **Runtime controls:** rate and cost limits, loop caps, context rot mitigation (periodic re-injection, fresh sessions for high-stakes), output filters, observability.
- ☐ **Privacy-specific controls:** refuse/ground on personal data queries, rectification capability, removal capability, data classification at the boundary, DPIA.
- ☐ **Process controls:** MCP review gate, AI-BOM, incident response playbook, regular review cadence.

Deliverable at end of Phase 3

A mitigation-to-threat traceability matrix. One row per threat, columns for the primary mitigation, the backstop, the owner, the deadline, and the decision type. This becomes the artefact the team actually ships.

Phase 4: Did we do a good enough job?

Time box: 15 minutes. Yes, really. Do not skip it.

This is the phase most sessions skip, and it is the phase the Manifesto puts the most weight on. It is where you make the explicit judgement call that the work is done enough to ship.

Per-question evidence checklist

Question 1 evidence.

- ☐ The DFD exists and the team agrees it reflects the shipping system.
- ☐ Trust boundaries, assets, actors, and data classifications are documented.

Question 2 evidence.

- ☐ Every threat is traceable to a DFD component.
- ☐ All four lenses (adversarial, structural, privacy, MCP) have been applied where relevant.
- ☐ Every threat has a system-specific example, not a generic one.
- ☐ The team can name threats they considered and rejected, not just ones they found.

Question 3 evidence.

- ☐ Every threat has a decision recorded.
- ☐ Every mitigation has an owner and a deadline.
- ☐ Every accepted risk has sign-off at the appropriate level.
- ☐ The traceability matrix exists.
- ☐ The "structural first" test has been applied.

Question 4 evidence.

- ☐ The threat model is versioned. This session produced version N.
- ☐ A trigger for the next review is agreed: a date, a feature, an incident type, or a dependency change.
- ☐ Someone outside the immediate team is scheduled to review the output.
- ☐ At least one design decision changed as a result of this session. If none did, the session was theatre.

The Manifesto values translation

Ask these out loud, at the end of the session, to the team:

- "Did we change the design, or are we just producing a document?"
- "Did the people who will build this participate, and did they argue?"
- "Do we understand the threats better than we did at the start?"
- "Have we produced owners, deadlines, and design changes, or just a list?"
- "Do we have a plan for the next review?"

If the answer to any of these is no, the session is not finished.

Red flags at this phase

- "Everything is accepted." Nobody accepts everything. Either the system is not being honest about risk, or the team is checking out.
- "We will fix it later." Later never comes. If it cannot be fixed before ship, it is a named accepted risk with sign-off, or it is a blocker.
- "The vendor takes care of that." Write down which control you are assuming the vendor provides, and what happens if they do not.
- "No design decisions changed." The session was theatre. Decide whether to continue, reconvene, or escalate.

Post-session

Immediate (within 24 hours)

- ☐ Circulate the DFD, threat list, mitigation matrix, and decisions to everyone who attended.
- ☐ Log mitigations in whatever the team actually tracks work in (Jira, Linear, GitHub issues). Not a document nobody will reopen.
- ☐ Record the version number of the threat model and the date of the session.
- ☐ Set the trigger for the next review.
- ☐ Flag anything that needs escalation: unresolved accepted risks, disagreements that could not be resolved in the room, blockers.

Within a week

- ☐ External review. Someone who was not in the room reads the output and tries to poke holes.
- ☐ Any blockers are raised to the appropriate level.
- ☐ The DPIA is updated if privacy is in scope.
- ☐ The AI-BOM is updated.

Ongoing

- ☐ The threat model is re-run on the trigger. Typical triggers: new feature, new dependency, new MCP server, new agent, incident affecting similar systems, regulatory change, quarterly cadence minimum.
- ☐ Mitigations are tracked to completion, not just logged.
- ☐ Accepted risks are reviewed on a schedule, because "accepted" does not mean "forever".

Quick-reference: The five questions you must ask every session

If you remember nothing else, ask these five questions on every AI system you threat model. They catch most of what the long checklists are trying to catch.

1. **What does the model read?** Every channel through which text reaches the model is a potential injection vector. (LLM01)
2. **What can the model do?** Every tool, every action with a side effect, every downstream consumer of model output. (LLM05, LLM06, ASI02, ASI05)
3. **Whose credentials is it using?** Every agent, every MCP server, every service account. (LLM06, ASI03)
4. **What happens when it is wrong?** Hallucinations, context rot, cascading failures. (LLM09, structural hazards)
5. **Can we delete personal data from every store it touches?** Memory, RAG, logs, caches. (LINDDUN unintervenability, T.R.I.M. Retention/Removal, GDPR Article 16/17)
6. **If the model's refusal or classification is the security guarantee, what replaces it when the subspace problem or a transfer attack bypasses it?** If there is no answer, the design is wrong.

A session that answers these five questions well, for any AI system, is already better than 90 percent of what is currently being done.

Summary

By the end of a session run using this runbook, you should have:

- A DFD of the system as it will actually ship.
- A threat list covering adversarial, structural, privacy, and MCP concerns, each with system-specific examples.
- A mitigation matrix with owners, deadlines, and decisions.
- Explicit evidence that the four Manifesto questions have been answered.
- A versioned artefact and a trigger for the next review.
- At minimum one concrete design change that would not have happened without the session.

That is what good enough looks like.

Final thoughts

The thing to remember as a facilitator is that the output is not the document, it is the shared understanding and the decisions. Teams that walk out of a threat modelling session with a better grasp of their own system and a short list of changes they have committed to make are doing it right, even if the document is messy. Teams that walk out with a perfectly formatted report and no changes are doing it wrong, even if the document is beautiful.

Run the session, change the design, ship something safer. That is the whole job.

Happy threat modelling.

Tags: threat-modelling, runbook, facilitator-guide, AI-security, LLM, agentic-AI, MCP, LINDDUN, T.R.I.M., Threat-Modeling-Manifesto